

Министерство науки и высшего образования РФ
Пензенский государственный университет
Кафедра «Вычислительная техника»

Отчет
по лабораторной работе № 3
по курсу «Арифметические и логические основы вычислительной
техники»
на тему «Динамические списки»

Выполнили
студенты группы 24ВВВ2:

Макаров Я.С.

Бегичев Д.Д.

Родионов К.Е.

Приняли

Юрова О.В.

Деев М.В.

Пенза, 2025

Цель работы

Изучить и практически реализовать абстрактные типы данных — стек, очередь и приоритетную очередь — на основе динамической структуры данных (односвязного списка).

Лабораторное задание

1. Приоритетная очередь: Элементы добавляются в список в соответствии с их приоритетом. Элемент с более высоким приоритетом должен располагаться в списке перед элементом с более низким.
2. Очередь: Новые элементы добавляются в конец списка, а извлекаются из его начала.
3. Стек: Новые элементы добавляются в начало списка и извлекаются также из начала.

Описание метода решения

Метод решения состоит в использовании **односвязного списка** как основы для реализации всех трех структур. Разница заключается только в логике добавления и удаления элементов:

- **Приоритетная очередь:** Элемент вставляется в отсортированную по приоритету позицию. Извлекается всегда из начала.
- **Очередь:** Элемент добавляется в конец списка, а извлекается из начала.
- **Стек:** Элемент добавляется и извлекается из начала списка.

Псевдокод

1-2 задание

алг приоритетная_очередь_людей

дано

- Структура данных узел (имя, приоритет, указатель на следующий узел)
- Указатели на голову (начало) и хвост (конец) очереди

надо

- Реализовать меню для управления очередью: добавление, просмотр, удаление по номеру, очистка

нач

// Функция для создания нового узла и ввода данных

функция получить_узел()

нач

 p := выделить память под узел
 если память не выделена **тогда**
 вывод "Ошибка памяти"
 завершение программы
 конец если

 ввод имя

если имя пустое **тогда**
 освободить память p
 вернуть пусто

конец если

 p.имя := имя

 ввод приоритет

если приоритет не число **тогда**
 освободить память p
 вернуть пусто

конец если

 p.приоритет := приоритет

 p.след := пусто

вернуть p

кон

// Процедура добавления человека с сохранением сортировки по приоритету

процедура добавить_человека()

нач

 p := вызов получить_узел()

если p = пусто **тогда вернуть**

если голова = пусто **тогда**

голова := р

хвост := р

иначе

текущий := голова

предыдущий := пусто

нц пока текущий $\neq$ пусто **и** текущий.приоритет $\leq$ р.приоритет

предыдущий := текущий

текущий := текущий.след

кц

если предыдущий = пусто **тогда** // Вставка в начало

р.след := голова

голова := р

иначе если текущий = пусто **тогда** // Вставка в конец

хвост.след := р

хвост := р

иначе // Вставка в середину

предыдущий.след := р

р.след := текущий

конец если

конец если

вывод "Человек добавлен"

кон

// Процедура для вывода всей очереди

процедура показать_очередь()

нач

текущий := голова

если текущий = пусто **тогда**

вывод "Очередь пуста"

иначе

позиция := 1

нц пока текущий $\neq$ пусто

вывод позиция, текущий.имя, текущий.приоритет

текущий := текущий.след

позиция := позиция + 1

кц

конец если

кон

// Процедура удаления элемента по его порядковому номеру

процедура удалить_по_номеру()

нач

если голова = пусто **тогда** вывод "Очередь пуста", **вернуть**

ввод позиция

текущий := голова, предыдущий := пусто, счетчик := 1

нц пока текущий $\neq$ пусто **и** счетчик < позиция

 предыдущий := текущий

 текущий := текущий.след

 счетчик := счетчик + 1

кц

если текущий = пусто **тогда** вывод "Номер не найден", **вернуть**

если предыдущий = пусто **тогда** голова := текущий.след

иначе предыдущий.след := текущий.след

если текущий.след = пусто **тогда** хвост := предыдущий

 вывод "Человек удален"

 освободить память текущий

кон

// Процедура для полного удаления всех элементов очереди

процедура очистить_очередь()

нач

 текущий := голова

нц пока текущий $\neq$ пусто

 след := текущий.след

 освободить память текущий

 текущий := след

кц

 голова := пусто

 хвост := пусто

 вывод "Очередь очищена"

кон

// Основной блок программы с меню

основная_программа

нач

нц пока истина

 вывод меню (1-добавить, 2-показать, 3-удалить, 4-очистить, 0-выход)

 ввод выбор

выбор выбор **из**

случай 1: вызов добавить_человека()

случай 2: вызов показать_очередь()

случай 3: вызов удалить_по_номеру()

случай 4: вызов очистить_очередь()

случай 0: вызов очистить_очередь(), завершение программы

иначе: вывод "Неверный выбор"

конец выбора

кц

кон

3 задание

алг стек_на_связном_списке

дано

- Структура данных узел (информационное поле, указатель на следующий узел)
- Указатель голова на вершину стека

надо

- Реализовать меню для управления стеком: добавить, удалить, посмотреть верхний, вывести всё, выйти

нач

// Функция проверки, пуст ли стек

функция пуст_ли()

вернуть (голова = пусто)

// Процедура добавления элемента в стек (push)

процедура добавить(значение)

нач

р := выделить память под узел

если память не выделена **тогда**

вывод "Ошибка памяти", завершение программы

конец если

р.инф := значение

р.след := голова

голова := р

вывод "Добавлен в стек:", значение

кон

// Функция удаления элемента из стека (pop)

функция удалить()

нач

если пуст_ли() **тогда**

вывод "Стек пуст"

вернуть пусто

конец если

temp := голова

голова := голова.след

значение := temp.инф

освободить память temp

```

        вернуть значение
    кон

// Функция просмотра верхнего элемента (peek)
функция посмотреть_верхний()
    нач
        если пуст_ли() тогда
            вывод "Стек пуст"
            вернуть пусто
        конец если
        вернуть голова.инф
    кон

// Процедура вывода содержимого стека
процедура показать_стек()
    нач
        р := голова
        если пуст_ли() тогда
            вывод "Стек пуст"
            вернуть
        конец если
        вывод "Содержимое стека (верх -> низ):"
        нц пока р ≠ пусто
            вывод р.инф
            р := р.след
        кц
    кон

// Процедура полной очистки стека
процедура очистить_стек()
    нач
        нц пока не пуст_ли()
            temp := голова
            голова := голова.след
            освободить память temp
        кц
    кон

// Основной блок программы с меню
основная_программа
    нач
        нц пока истина
            вывод меню (1-добавить, 2-удалить, 3-посмотреть, 4-вывести, 5-
выход)
            ввод выбор
            выбор выбор из
                случай 1: ввод значение, вызов добавить(значение)

```


случай 2: результат :=
вызов удалить(), **если** результат ≠ пусто **тогда** вывод "Удалён:", результат
случай 3: результат :=
вызов посмотреть_верхний(), **если** результат ≠ пусто **тогда** вывод
"Верхний:", результат
случай 4: вызов показать_стек()
случай 5: вызов очистить_стек(), завершение программы
иначе: вывод "Неверный выбор"
конец выбора
кц
кон

Листинг

1-2 задание

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define _CRT_SECURE_NO_WARNINGS

struct node {
    char name[256];
    int priority;
    struct node* next;
};

struct node* head = NULL;
struct node* last = NULL;

struct node* get_struct(void) {
    struct node* p = NULL;
    char name[256];
    int priority;

    if ((p = (struct node*)malloc(sizeof(struct node))) ==
        NULL) {
        printf("Ошибка при распределении памяти\n");
        exit(1);
    }

    printf("Введите имя человека: ");
    scanf("%255s", name);
    if (strlen(name) == 0) {
        printf("Запись не была произведена\n");
        free(p);
    }
}
```

```

        return NULL;
    }
    strcpy(p->name, name);

    printf("Введите приоритет (целое число, чем меньше - тем
выше приоритет): ");
    if (scanf("%d", &priority) != 1){
        printf("Ошибка ввода приоритета\n");
        free(p);
        return NULL;
    }
    p->priority = priority;

    p->next = NULL;

    return p;
}

void add_person(void){
    struct node* p = NULL;
    struct node* current = NULL;
    struct node* prev = NULL;

    p = get_struct();
    if (p == NULL){
        return;
    }

    if (head == NULL){
        head = p;
        last = p;

        printf("Человек '%s' с приоритетом %d добавлен в
очередь\n", p->name, p->priority);
    }
}

```

```

        return;
    }

    current = head;
    prev = NULL;

    while (current != NULL && current->priority <= p->priority){
        prev = current;
        current = current->next;
    }

    if (prev == NULL){
        p->next = head;
        head = p;
    }
    else if (current == NULL){
        last->next = p;
        last = p;
    }
    else{
        prev->next = p;
        p->next = current;
    }

    printf("Человек '%s' с приоритетом %d добавлен в очередь\n",
p->name, p->priority);
}

void display_queue(void){
    struct node* current = head;

    if (head == NULL){

```

```

        printf("Очередь пуста\n");
        return;
    }

    printf("\n Текущее состояние очереди \n");
    int position = 1;
    while (current != NULL) {
        printf("%d. %s (приоритет: %d)\n", position, current->name, current->priority);
        current = current->next;
        position++;
    }
    printf("\n");
}

void remove_by_position(void) {
    if (head == NULL) {
        printf("Очередь пуста\n");
        return;
    }

    int position;
    display_queue();
    printf("Введите номер человека для удаления: ");

    if (scanf("%d", &position) != 1 || position < 1) {
        printf("Неверный номер\n");
        return;
    }

    struct node* current = head;
    struct node* prev = NULL;

```

```

int current_pos = 1;

while (current != NULL && current_pos < position){
    prev = current;
    current = current->next;
    current_pos++;
}

if (current == NULL){
    printf("Человек с номером %d не найден\n", position);
    return;
}

if (prev == NULL){
    head = current->next;
    if (head == NULL){
        last = NULL;
    }
}
else{
    prev->next = current->next;
    if (current->next == NULL){
        last = prev;
    }
}

printf("Человек '%s' (приоритет: %d) удален из очереди\n",
current->name, current->priority);
free(current);
}

void clear_queue(void){

```

```

    struct node* current = head;
    struct node* next;

    while (current != NULL){
        next = current->next;
        free(current);
        current = next;
    }

    head = NULL;
    last = NULL;
    printf("Очередь очищена\n");
}

void menu(void){
    int choice;

    while (1){
        printf("\nПРИОРИТЕТНАЯ ОЧЕРЕДЬ ЛЮДЕЙ \n");
        printf("1. Добавить человека в очередь\n");
        printf("2. Просмотреть очередь\n");
        printf("3. Удалить человека по номеру\n");
        printf("4. Очистить очередь\n");
        printf("0. Выход\n");
        printf("Выберите действие: ");

        if (scanf("%d", &choice) != 1){
            printf("Ошибка ввода\n");
            while (getchar() != '\n');
            continue;
        }
    }
}

```

```

switch (choice){
case 1:
    add_person();
    break;

case 2:
    display_queue();
    break;

case 3:
    remove_by_position();
    break;

case 4:
    clear_queue();
    break;

case 0:
    clear_queue();
    printf("Выход из программы\n");
    return;

default:
    printf("Неверный выбор\n");
}
}

int main(void){
    system("chcp 1251");
    system("cls");
    menu();
}

```



```
return 0;
```

```
}
```

2-ое задание

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <windows.h>
#include <locale.h>

struct node {
    char inf[256];
    struct node* next;
};

struct node* head = NULL;

int isEmpty() {
    return head == NULL;
}

void push(char* value) {
    struct node* p = (struct node*)malloc(sizeof(struct node));
    if (!p) {
        printf("ОШИБКА: не удалось выделить память\n");
        exit(1);
    }
    strcpy_s(p->inf, sizeof(p->inf), value);
    p->next = head;
    head = p;
    printf("Добавлен в стек: %s\n", value);
}

char* pop() {
    if (isEmpty()) {
```

```

        printf("Стек пуст\n");
        return NULL;
    }
    struct node* temp = head;
    head = head->next;
    char* value = _strdup(temp->inf);
    free(temp);
    return value;
}

char* peek() {
    if (isEmpty()) {
        printf("Стек пуст\n");
        return NULL;
    }
    return head->inf;
}

void printStack() {
    struct node* p = head;
    if (isEmpty()) {
        printf("Стек пуст\n");
        return;
    }
    printf("Содержимое стека (верх -> низ):\n");
    while (p) {
        printf("%s\n", p->inf);
        p = p->next;
    }
}

void clearStack() {

```

```

while (!isEmpty()) {
    struct node* tmp = head;
    head = head->next;
    free(tmp);
}

}

int main() {
    setlocale(LC_ALL, "Russian");

    int choice;
    char value[256];

    while (1) {
        printf("\nМеню:\n");
        printf("1. Добавить элемент\n");
        printf("2. Удалить элемент\n");
        printf("3. Посмотреть верхний элемент\n");
        printf("4. Вывести стек\n");
        printf("5. Выход\n");
        printf("Номер: ");
        scanf_s("%d", &choice);

        switch (choice) {
            case 1:
                printf("Введите значение: ");
                scanf_s("%255s", value, (unsigned)_countof(value));
                push(value);
                break;
            case 2: {
                char* res = pop();
                if (res) {

```

```
        printf("Удалён элемент: %s\n", res);
        free(res);
    }
    break;
}
case 3:
    if (peek()) {
        printf("Верхний элемент стека: %s\n", peek());
    }
    break;
case 4:
    printStack();
    break;
case 5:
    clearStack();
    printf("Выход из программы\n");
    return 0;
default:
    printf("Неверный выбор\n");
}
}
}
```

Результат работы программы

1-2 задание

```
ПРИОРИТЕТНАЯ ОЧЕРЕДЬ ЛЮДЕЙ
1. Добавить человека в очередь
2. Просмотреть очередь
3. Удалить человека по номеру
4. Очистить очередь
0. Выход
Выберите действие: 1
Введите имя человека: Ярослав
Введите приоритет (целое число, чем меньше - тем выше приоритет): 5
Человек 'Ярослав' с приоритетом 5 добавлен в очередь
```

```
ПРИОРИТЕТНАЯ ОЧЕРЕДЬ ЛЮДЕЙ
1. Добавить человека в очередь
2. Просмотреть очередь
3. Удалить человека по номеру
4. Очистить очередь
0. Выход
Выберите действие: 2

Текущее состояние очереди
1. Ярослав (приоритет: 1)
2. Денис (приоритет: 2)
3. Кирилл (приоритет: 3)
4. Ярослав (приоритет: 5)
5. денис (приоритет: 10)
```

3 задание

```
Меню:
1. Добавить элемент
2. Удалить элемент
3. Посмотреть верхний элемент
4. Вывести стек
5. Выход
Номер: 4
Содержимое стека (верх -> низ):
7
6
5
4
3
2
1
```

```
Содержимое стека (верх -> низ):  
1000000000000  
100000  
1000  
100  
7  
6  
5  
4  
3  
2  
1
```

Вывод

В ходе данной работы были реализованы стек, очередь и приоритетная очередь на основе односвязного списка. Это позволило на практике убедиться, что путем изменения логики операций добавления и удаления элементов можно на базе одной и той же фундаментальной структуры данных создавать различные абстрактные типы данных. В процессе выполнения были закреплены навыки работы с динамической памятью и указателями.